

E.1 (K, L) in LSH

In each hash table, we use K hash functions to compute the hash code of k and q . In Simhash (Charikar, 2002), the hashing we use in MAGICPIG, the hash functions are random projections. With K random projections, we are able to partition the space (in our problem, the space is R^{128}) into 2^K subspace. If and only if k and q fall in the same subspace, we say they collide in this hash table. We have L hash tables in total. In MAGICPIG, if and only if k and q collide in at least two hash tables, k is sampled by q . Here are some intuitions about how (K, L) will influence the LSH sampling in MAGICPIG.

- If K is too small, then we cannot partition the space well; we will sample too many ks , which might be far away from q (in the attention problem, this means their inner production is small), increasing computation cost.
- On the other hand, if K is too large, although the quality of sampled ks will be better, the collision probability in each table will be small; thus, the number of the sampled ks will be reduced. We need to increase L to ensure that a certain number of keys are sampled and involved in the computation. However, increasing (K, L) too much will bring more memory overhead on CPU DRAM since we build L hash tables for each key-value head.

Thus, (K, L) is important because it balances computation cost, overhead, and sampling quality (which determines accuracy). Tuning (K, L) is necessary in LSH (Lv et al., 2017; Slaney et al., 2012).

E.2 (K, L) and memory overhead

(K, L) will change two overheads brought by MAGICPIG: the memory occupied by hash tables on the CPU and extra computation for random projections (hash functions) on the GPU (as shown in Table 8).

Table 8 The overhead of Locality sensitive hashing during decoding. We report the size of random projectors (on GPU) and hash tables (on CPU), the computation overhead **CO** (refers to the ratio between computation introduced by random projections in LSH and the computation of the original model’s linear projections (e.g., W_Q, W_K, W_V , and MLP)). Notice that when the context length exceeds 64K, we need to use 32-bit integers to store the indices for the KV cache in hash tables. Llama-3.1-8B/70B-Instruct (Dubey et al., 2024) and Code-Llama-34b-16K Rozière et al. (2024) use group query attention, thus the sizes of hash tables are reduced.

Models	(K, L)	Context length	Projectors	Hash tables	CO
<i>Llama-3.1-8B-Instruct</i>	(10, 150)	96K	384KB	14GB	3.8%
<i>Llama-3.1-8B-Instruct</i>	(11, 300)	96K	825KB	28GB	8.5%
<i>Llama-3.1-8B-Instruct</i>	(10, 150)	64K	384KB	4.7GB	3.8%
<i>Llama-3.1-70B-Instruct</i>	(10, 150)	64K	384KB	11.8GB	1.8%
<i>Code-Llama-13b-16K</i>	(10, 150)	16K	384KB	7.3GB	5.2%
<i>Code-Llama-34b-16K</i>	(10, 150)	16K	384KB	1.8GB	2.2%

LLM decoding is a memory-bandwidth-bound process and the majority of time is spent loading the data (parameters/KV cache) to GPU cores rather than actually doing the computation (Miao et al., 2023; Zhang et al., 2023a; Chen et al., 2024). Besides, the time-consuming part, i.e., the long-context attention computation, is moved to the CPU. Thus, the 1.8% $\sim$ 8.5% extra computation on GPU will only make a minor difference in execution time. However, the enlarged size of hash tables prevents us from always increasing (K, L) to get more accurate results.

As shown in Table 8, under the same (K, L) , the memory overhead of hash tables grows linearly with context length and the total number of key-value heads in models (which is determined by model sizes).

E.3 (K, L) and computation cost/budget

In summary, increasing K will make the budget⁵ smaller, and increasing L will increase the budget.

Theoretically, as introduced in Section 4.3, in our approach, the key k_i is sampled only if at least two hash tables exist where k_i shares the hash value with query q . With the assumption that k_i is well-distributed (In

⁵Cost₂ in Tables 1 to 3

each hash table out of L , each hash value corresponds to roughly the same number of k_i s), the ratio of retrieved k_i s can be estimated with

$$\mathcal{B}/n = 1 - (1 - 0.5^K)^L - L \times 0.5^K (1 - 0.5^K)^{L-1} \quad (21)$$

where n is the context length. Here, we estimate the collision probability of k_i and q in a single hash table as 0.5^K .

Empirically, the ratio of retrieved keys and values ($\mathcal{B}/n$) might differ from the above estimation since the data is not perfectly distributed. We present the empirically measured budget in Table 9.

Table 9 Empirical measured budget/cost for different (K, L).

K / L	75	100	120	150	200	300
7	14%	21%	27%	35%	48%	66%
8	5%	8%	11%	15%	22%	36%
9	1.6%	2.7%	4%	5.4%	8.5%	15.4%
10	0.5%	0.9%	1.5%	2%	3%	6%
11	0.15%	0.3%	0.5%	0.6%	1%	2%

E.4 (K, L) and accuracy

There are no naive relations between (K, L) and downstream accuracies since (K, L) not only influences sampling quality but also the computation budget. One safe way to discuss the relation between (K, L) and accuracy is: Fixing the computation budget, larger (K, L) will potentially produce higher accuracy, since the sampling quality is higher. Our experimental results show that,

- Increasing (K, L) can significantly improve accuracy in relatively longer contexts Table 10.

Table 10 We show the effectiveness of larger hash tables for longer contexts by evaluating MegaBeam-Mistral-7B-512K on RULER (Hsieh et al., 2024). With the same computation cost ($\sim 2\%$), config (11, 300) achieves higher accuracy compared to (10, 150).

(K, L)	16K	128K	256K
Full	91.7	83.7	82.5
(10, 150)	89.8	80.7	79.0
(11, 300)	90.6	83.3	81.9

- Same set of (K, L) can generalize to larger LLMs Table 11.

Table 11 8B and 70B models on RULER (Hsieh et al., 2024) 64K.

Models/Config	Full	(10, 150)	(10, 135)	(9, 120)	(9, 110)
Llama-3.1-8B-Instruct	86.1	84.8	83.6	84.7	84.7
Llama-3.1-70B-Instruct	89.2	87.5	86.7	88.8	88.4

E.5 How to select (K, L)

Finding the optimal (K, L) for high accuracy as well as efficiency is a long-standing problem in LSH. Similar to the traditional hyper-parameter tuning process in machine learning, K, and L are configured offline based on data subsets. In LSH, K is a more sensitive hyper-parameter than L. A slight change of K can drastically influence the number of retrieved items (i.e., budget/cost) and quality. In MAGICPIG, K=8-10 is manually determined by ablations on small-scale tasks and found to be effective across various models and tasks; then, we adjust L to obtain the wanted computation cost/budget.

Here, we present two ablations to demonstrate the selection of K in Tables 12 and 13.

Table 12 Fixing the budget/cost to 4%, we ablation the performance of different (K, L) on RULER (Hsieh et al., 2024) 16K.

Models/Config	Full	(10, 240)	(9, 120)	(8, 65)	(7, 35)
Llama-3.1-8B-Instruct	94.2	94.2	92.8	92.3	88.5

Table 13 Fixing L as 120, we ablation the performance of different K on RULER (Hsieh et al., 2024) 16K for Llama-3.1-8B-Instruct.

(K, L)	Full	(10, 120)	(9, 120)	(8, 120)	(7, 120)
Cost	1.0	0.012	0.04	0.11	0.27
Accuracy	94.2	92.8	92.8	94.1	94.3

If we want the computation cost to be below 5% and L below 200 (to reduce memory overhead in the CPU), then K=8-10 is a reasonable choice. Unlike K, L is not that sensitive. We select L based on the following principle after determining K: we can allow the computation cost to be smaller for larger K since the sampling is more precise. This is why we choose to use (8, 75), (9, 120), and (10, 150).

It’s worth pointing out that tuning (K, L) is a challenging and long-standing problem in LSH, and we only give an example of practice in MAGICPIG. More advanced hashing algorithms (such as Cross-polytope (Andoni et al., 2015) or data-dependent ones (Andoni and Razenshteyn, 2015)) can improve the trade-off between memory overhead and accuracy. We leave it as a future direction.

F TopK vs. Sampling

In this section, we provide an intuitive understanding of how sampling can work better than TopK. TopK only captures the ranking information when estimating attention output. In contrast, sampling considers the entire data distribution (i.e., the attention score after Softmax).

Here is an example. Imagine a zoo with 100 animals: 10 elephants, 10 pigs, 10 tigers, and 70 other unique animals. The daily food consumption for each group is as follows:

- **Elephants:** 50 lb/day each
- **Pigs:** 20 lb/day each
- **Tigers:** 10 lb/day each
- **Other unique animals:** 1 lb/day each

To compute the true average daily food consumption per animal in the zoo:

$$\text{True Average} = \frac{(10 \times 50) + (10 \times 20) + (10 \times 10) + (70 \times 1)}{100} = 8.7 \text{ lb.}$$

If we use a **Top-K approach** (e.g., selecting the top 10 animals based on the numbers of animals), we include elephants, pigs, tigers, and 7 randomly selected animals from the unique ones. The estimated average is:

$$\text{TopK Average} = \frac{(10 \times 50) + (10 \times 20) + (10 \times 10) + (7 \times 1)}{37} = 22 \text{ lb.}$$

This overestimates the average because it disproportionately weights high-consumption animals.

Instead, we perform **sampling with replacement** from the animal distribution, proportional to their numbers. The probabilities for each group are:

$$\text{Sampling Probabilities} = [0.1, 0.1, 0.1, 0.01 \times 70],$$

where 0.1 represents the probabilities for elephants, pigs, and tigers (10/100 each), and 0.01 corresponds to each unique animal (1/100).